DATA STRUCTURES AND ALGORITHMS

Comprehensive Study Material

Last Updated: June 30, 2026

TABLE OF CONTENTS

1. ARRAYS AND STRINGS

Arrays: Arrays are ordered collections of elements stored in contiguous memory. They provide O(1) random access but O(n) insertion/deletion. Two-pointer technique is useful for problems like palindromes and pair sums.

Two-Pointer Technique: Uses two pointers moving from opposite ends to solve problems efficiently. Common applications: finding pairs with target sum, checking palindromes, merging sorted arrays.

Sliding Window: Maintains a window of elements and slides it across the array. Useful for subarray/substring problems. Time complexity is reduced from $O(n^2)$ to O(n).

Strings: Strings are sequences of characters. String manipulation includes reversal, rotation, anagram checking, and pattern matching. Immutability in many languages affects performance.

2. LINKED LISTS

Structure: Linked lists consist of nodes with data and pointers to the next node. Unlike arrays, they provide O(n) random access but O(1) insertion/deletion when pointer is known.

Operations: Traversal, insertion, deletion, reversal. Common problems include detecting cycles (Floyd's algorithm), finding middle element, merging sorted lists.

Cycle Detection: Floyd's Cycle Detection Algorithm uses slow and fast pointers. If they meet, a cycle exists. The algorithm runs in O(n) time with O(1) space.

Variations: Singly linked lists, doubly linked lists, and circular linked lists serve different purposes.

3. STACKS AND QUEUES

Stacks (LIFO): Last-In-First-Out data structure. Push adds elements, pop removes from the top. Applications include function call management, undo/redo, expression evaluation, backtracking.

Queues (FIFO): First-In-First-Out data structure. Enqueue adds elements at rear, dequeue removes from front. Applications include BFS, printer queues, message passing systems.

Priority Queues: Elements have associated priorities. Highest priority element is dequeued first. Implemented using heaps. Applications include task scheduling, Dijkstra's algorithm.

Deques: Double-ended queues allow insertion and deletion from both ends. Used in sliding window maximum problems.

4. TREES

Binary Trees: Each node has at most two children (left and right). Traversal methods: inorder (left-root-right), preorder (root-left-right), postorder (left-right-root), level-order.

Binary Search Trees: Ordered binary trees where left child < parent < right child. Enable efficient searching, insertion, deletion in O(log n) average time. Unbalanced trees degrade to O(n).

Balanced Trees: AVL trees and Red-Black trees maintain balance to ensure O(log n) operations. Self-balancing through rotations during insertions and deletions.

Tries: Tree-like data structures for storing strings. Prefix-based searches run in O(m) where m is string length. Used in autocomplete and spell checking.

Heaps: Complete binary trees that satisfy heap property. Min-heap: parent < children. Max-heap: parent > children. Support efficient priority queue operations.

5. GRAPHS

Representation: Adjacency matrix (2D array) or adjacency list (linked lists). Lists are space-efficient for sparse graphs, matrices for dense graphs.

Breadth-First Search (BFS): Explores nodes level by level using a queue. Time O(V+E), space O(V). Finds shortest path in unweighted graphs.

Depth-First Search (DFS): Explores nodes using recursion or stack. Time O(V+E), space O(V). Useful for topological sorting, cycle detection, connected components.

Dijkstra's Algorithm: Finds shortest path from source to all vertices in weighted graphs. Time O((V+E)logV) with priority queue. Cannot handle negative weights.

Floyd-Warshall: All-pairs shortest path algorithm. Time $O(V^3)$, handles negative weights. Dynamic programming approach.

6. SORTING ALGORITHMS

Bubble Sort: Repeatedly swaps adjacent elements if they're in wrong order. Time: $O(n^2)$, Space: O(1). Simple but inefficient for large datasets.

Quick Sort: Divide-and-conquer using pivot partitioning. Average O(n log n), worst $O(n^2)$. In-place sorting, excellent practical performance.

Merge Sort: Divide-and-conquer combining sorted subarrays. Time O(n log n), Space O(n). Stable sort, consistent performance regardless of input.

Heap Sort: Uses heap data structure. Time O(n log n), Space O(1). In-place but unstable sort.

7. SEARCHING ALGORITHMS

Linear Search: Scans entire array sequentially. Time O(n), works on unsorted data. Simple but inefficient for large datasets.

Binary Search: Divides sorted array in half repeatedly. Time O(log n), Space O(1) or O(log n) if recursive. Requires pre-sorted data.

Hash Tables: Map keys to values using hash function. Average O(1) insertion/deletion/lookup. Handles collisions via chaining or open addressing.

8. DYNAMIC PROGRAMMING

Concept: Solves problems by breaking them into overlapping subproblems. Memoization (top-down) or tabulation (bottom-up) avoids recomputation.

Common Problems: Fibonacci, Longest Common Subsequence (LCS), 0/1 Knapsack, Coin Change, Longest Increasing Subsequence (LIS).

0/1 Knapsack: Select items to maximize value within weight constraint. DP table of size O(n×W). Time O(n×W), Space O(n×W).

9. GREEDY ALGORITHMS

Concept: Makes locally optimal choices hoping to find global optimum. Works when problem has greedy choice property and optimal substructure.

Activity Selection: Select maximum non-overlapping activities. Sort by end time, greedily select. Time O(n log n).

Huffman Coding: Creates optimal prefix-free code for compression. Build binary tree bottom-up. Greedy approach guarantees optimality.

10. COMPLEXITY ANALYSIS

Big O Notation: Describes upper bound of algorithm complexity. Ignores constants and lower-order terms. Used for worst-case analysis.

Common Complexities: O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, $O(n^2)$ quadratic, $O(2^n)$ exponential, O(n!) factorial.

Space Complexity: Analyzes memory usage beyond input. Includes auxiliary space for data structures and recursion stack.

Trade-offs: Often optimize time at cost of space or vice versa. Choose based on constraints and requirements.